E6 — Gérer et faire évoluer un entrepôt de données

E6 — Gérer et faire évoluer un entrepôt de données

Certification Ingénieur en Infrastructure Massive de Données

Candidat : Mezine Elias

Bloc de compétences : Bloc 3 — Élaborer et maintenir un entrepôt de données

Livrable : E6 — Gérer et faire évoluer un entrepôt de données

---

Sommaire

Gérer l'entrepôt de données **3**
1. Journalisation . . . 3
   1.1 Catégorisation des logs . . . 3
   1.2 Logs dans GitHub Actions . . . 3
   1.3 Suivi via table de métadonnées dans Snowflake . . . 3
2. Système d'alerte . . . 4
3. Tâches de maintenance . . . 4
   3.1 Priorisation des tâches . . . 4
   3.2 Assignation des tâches . . . 5
4. Indicateurs de service . . . 6
   4.1 SLA Snowflake (fournisseur) . . . 6
   4.2 Tableau de bord et indicateurs de suivi . . . 6
5. Backups . . . 9
6. Documentation du cas d'usage de gestion . . . 10
   6.1 Intégration d'une nouvelle source de référence : `taxi_zone_lookup` . . . 10
   6.2 Évolution structurelle de l'entrepôt . . . 10
   6.3 Gestion des accès aux nouveaux objets . . . 11
   6.4 Espace de stockage . . . 11
   6.5 Capacité de calcul . . . 11
7. Conformité RGPD . . . 11
   7.1 Registre des traitements de données personnelles . . . 11
   7.2 Procédures de tri des données personnelles . . . 12
   7.3 Cookies de la documentation en ligne (Google Analytics) . . . 12

Implémenter des variations dans les dimensions de l'entrepôt de données **14**
1. Modélisation de l'entrepôt après évolution . . . 14

2. Types de variations (Slowly Changing Dimensions - SCD) 16
2.1 Rappel des types SCD 16
2.2 Choix des SCD par dimension 16
2.3 Scénarios hypothétiques pour dim_locations 16
3. Documentation de l'état actuel 16

Synthèse **18**

Gérer l'entrepôt de données

Contexte : Cette étude de cas suppose l'entrepôt de données NYC_TAXI_DW déjà existant et opérationnel comme présenté en E5.

1. Journalisation

1.1 Catégorisation des logs

Le système de journalisation utilise le module Python `logging` avec 5 niveaux de détail configurables via la variable GitHub Actions `LOGGER_LEVEL`, dont la valeur par défaut est `INFO`. Chaque niveau inclut les messages des niveaux de sévérité supérieure.

Niveau	Usage
CRITICAL	Défaillances système irrécupérables
ERROR	Erreurs critiques (échec de connexion, échec de chargement, valeurs invalides)
WARNING	Alertes non-bloquantes (aucun fichier à charger, fichier déjà référencé)
INFO	Informations de progression (début/fin d'étape, nombre de fichiers, lignes chargées)
DEBUG	Détails techniques (requêtes SQL exécutées, mises à jour des métadonnées). Active également les logs internes Snowflake, traces OpenTelemetry

1.2 Logs dans GitHub Actions

Chaque étape du workflow GitHub Actions produit des logs détaillés consultables dans l'interface GitHub :

- Logs de chaque étape du pipeline
- Conservation des logs selon la politique de rétention GitHub

1.3 Suivi via table de métadonnées dans Snowflake

La table `FILE_LOADING_METADATA` permet de suivre l'état de chaque fichier tout au long de l'ingestion. La colonne `FILENAME`, ajoutée lors du chargement `COPY INTO` dans la table RAW, établit le lien entre chaque ligne et son fichier source dans `FILE_LOADING_METADATA`. Le nom de la table est configurable via la variable GitHub Actions `METADATA_TABLE`.

Statut	Signification
`SCRAPED`	Fichier détecté, en attente d'upload
`STAGED`	Fichier uploadé dans le stage
`SUCCESS`	Chargement réussi
`FAILED_STAGE`	Échec de l'upload vers le stage
`FAILED_LOAD`	Échec du chargement en table

		FILE_NAME	YEAR	MONTH	ROWS_LOADED	LOAD_STATUS	LOAD_TIMESTAMP
1	ıta/yellow_tripdata_2025-01.parq...	yellow_tripdata_2025-01.parquet	2025	1	0	FAILED_LOAD	2025-12-30T12:03:07.730+0100
2	ıta/yellow_tripdata_2025-02.parq...	yellow_tripdata_2025-02.parquet	2025	2	0	FAILED_LOAD	2025-12-30T12:03:09.272+0100
3	ıta/yellow_tripdata_2025-03.parq...	yellow_tripdata_2025-03.parquet	2025	3	0	FAILED_LOAD	2025-12-30T12:03:10.590+0100
4	ıta/yellow_tripdata_2025-04.parq...	yellow_tripdata_2025-04.parquet	2025	4	0	FAILED_LOAD	2025-12-30T12:03:11.110+0100
5	ıta/yellow_tripdata_2025-05.parq...	yellow_tripdata_2025-05.parquet	2025	5	0	FAILED_LOAD	2025-12-30T12:03:12.006+0100
6	ıta/yellow_tripdata_2025-06.parq...	yellow_tripdata_2025-06.parquet	2025	6	0	FAILED_LOAD	2025-12-30T12:03:12.354+0100
7	ıta/yellow_tripdata_2025-07.parq...	yellow_tripdata_2025-07.parquet	2025	7	0	FAILED_LOAD	2025-12-30T12:03:12.709+0100
8	ıta/yellow_tripdata_2025-08.parq...	yellow_tripdata_2025-08.parquet	2025	8	0	FAILED_LOAD	2025-12-30T12:03:13.079+0100
9	ıta/yellow_tripdata_2025-09.parq...	yellow_tripdata_2025-09.parquet	2025	9	0	FAILED_LOAD	2025-12-30T12:03:13.460+0100
10	ıta/yellow_tripdata_2025-10.parq...	yellow_tripdata_2025-10.parquet	2025	10	0	FAILED_LOAD	2025-12-30T12:03:13.839+0100
11	ıta/yellow_tripdata_2025-11.parq...	yellow_tripdata_2025-11.parquet	2025	11	0	FAILED_LOAD	2025-12-30T12:03:14.220+0100

Figure 1: Table de métadonnées FILE_LOADING_METADATA

2. Système d'alerte

GitHub Actions envoie des **notifications email automatiques** en cas de :

- **Échec** du workflow (statut `failure`)
- **Annulation** du workflow (statut `cancelled`)

Figure 2: Alertes e-mail GitHub en cas d'échec de workflow

3. Tâches de maintenance

3.1 Priorisation des tâches

Les tâches sont priorisées de P1 à P4 selon leur impact sur les objectifs et exigences de maintenance.

Priorité	Tâche	Objectifs et exigences de maintenance	Fréquence	Rôle
P1 - Critique	Surveillance du pipeline	Délai d'exécution < 30 min et taux de succès du chargement 100 %	Mensuelle	Data Engineer
P1 - Critique	Résolution des fichiers en statut FAILED	Résolution des incidents < 48h	À chaque échec	Data Engineer
P2 - Haute	Vérification des tests dbt	100 % des tests passants	Mensuelle	Data Engineer
P2 - Haute	Vérification des backups	Politiques de backup actives et fonctionnelles	Mensuelle	Data Engineer / DBA
P3 - Moyenne	Mise à jour des dépendances	Dépendances à jour, aucune vulnérabilité critique	Trimestrielle	DevOps
P3 - Moyenne	Analyse CodeQL	Aucune vulnérabilité critique non résolue	À chaque PR	DevOps
P3 - Moyenne	Revue de la consommation du warehouse	Consommation de crédits dans les limites attendues	Mensuelle	DBA
P4 - Basse	Revue de la documentation	Documentation à jour	À chaque évolution	Data Engineer

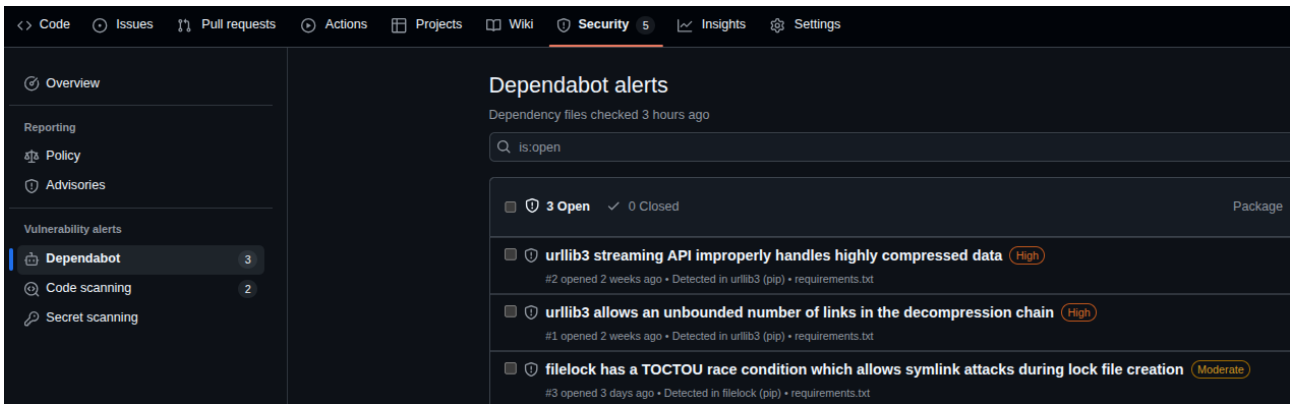


Figure 3: Alertes Dependabot

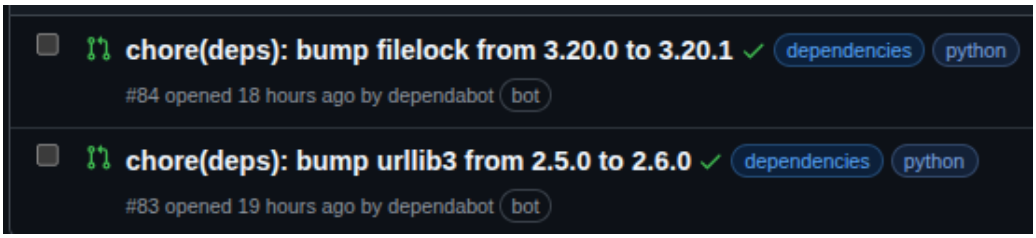


Figure 4: Pull Requests automatiques Dependabot

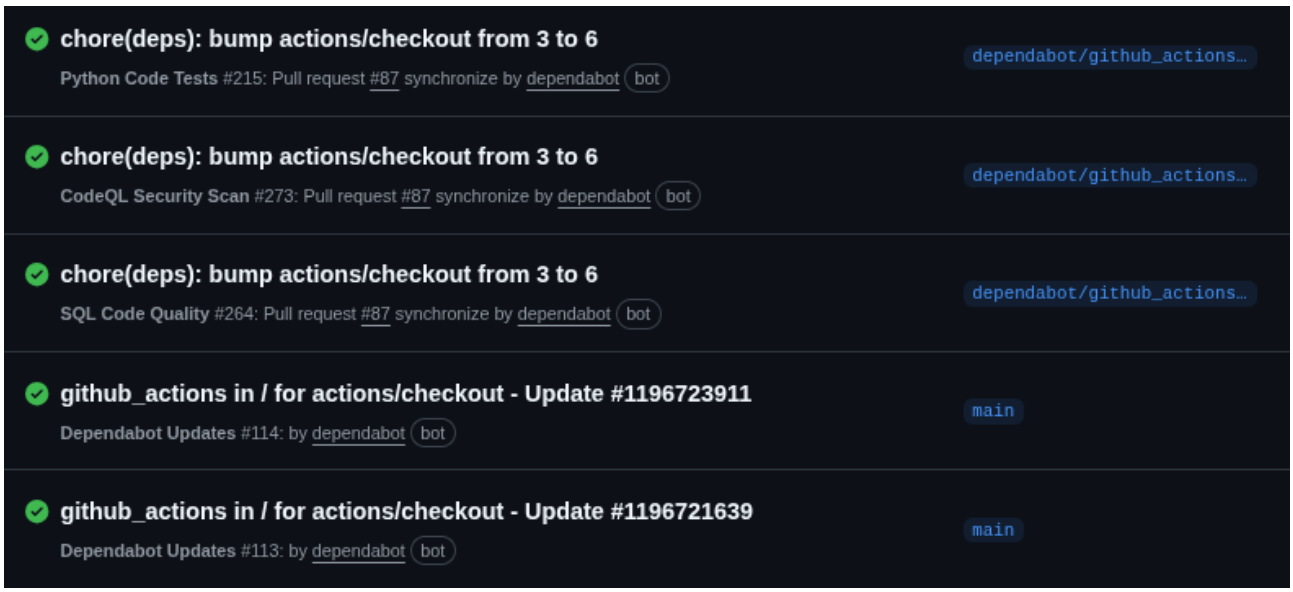


Figure 5: Mises à jour automatiques des GitHub Actions par Dependabot

3.2 Assignation des tâches

Dans ce projet, une seule personne porte les trois casquettes. Le **Data Engineer**, sous le compte `USER_DEV` avec le rôle `TRANSFORMER`, assure le pipeline, la résolution des échecs, les tests et la documentation. Le **DBA / Admin Snowflake** exécute les tâches d'administration via les rôles système Snowflake SYSADMIN, SECURITYADMIN et ACCOUNTADMIN : gestion des rôles et accès, backup policies, suivi de la consommation. Le **DevOps** gère les workflows GitHub Actions, les dépendances via Dependabot et la sécurité via CodeQL.

4. Indicateurs de service

4.1 SLA Snowflake (fournisseur)

Deux engagements s'appliquent simultanément sur la disponibilité mensuelle du service :

- **Seuil 99,9 %** : déclenche lorsque la durée de panne dépasse 43 minutes ou que le taux d'erreurs dépasse 1 %. Une seule occurrence dans le mois suffit à violer cet engagement.
- **Seuil 99,99 %** : couvre les pannes courtes et intenses, entre 4 et 43 minutes avec un taux d'erreurs supérieur à 10 %. Chaque panne de ce type est comptabilisée ; si leur cumul mensuel dépasse 43 minutes, l'engagement 99,9 % est alors violé.

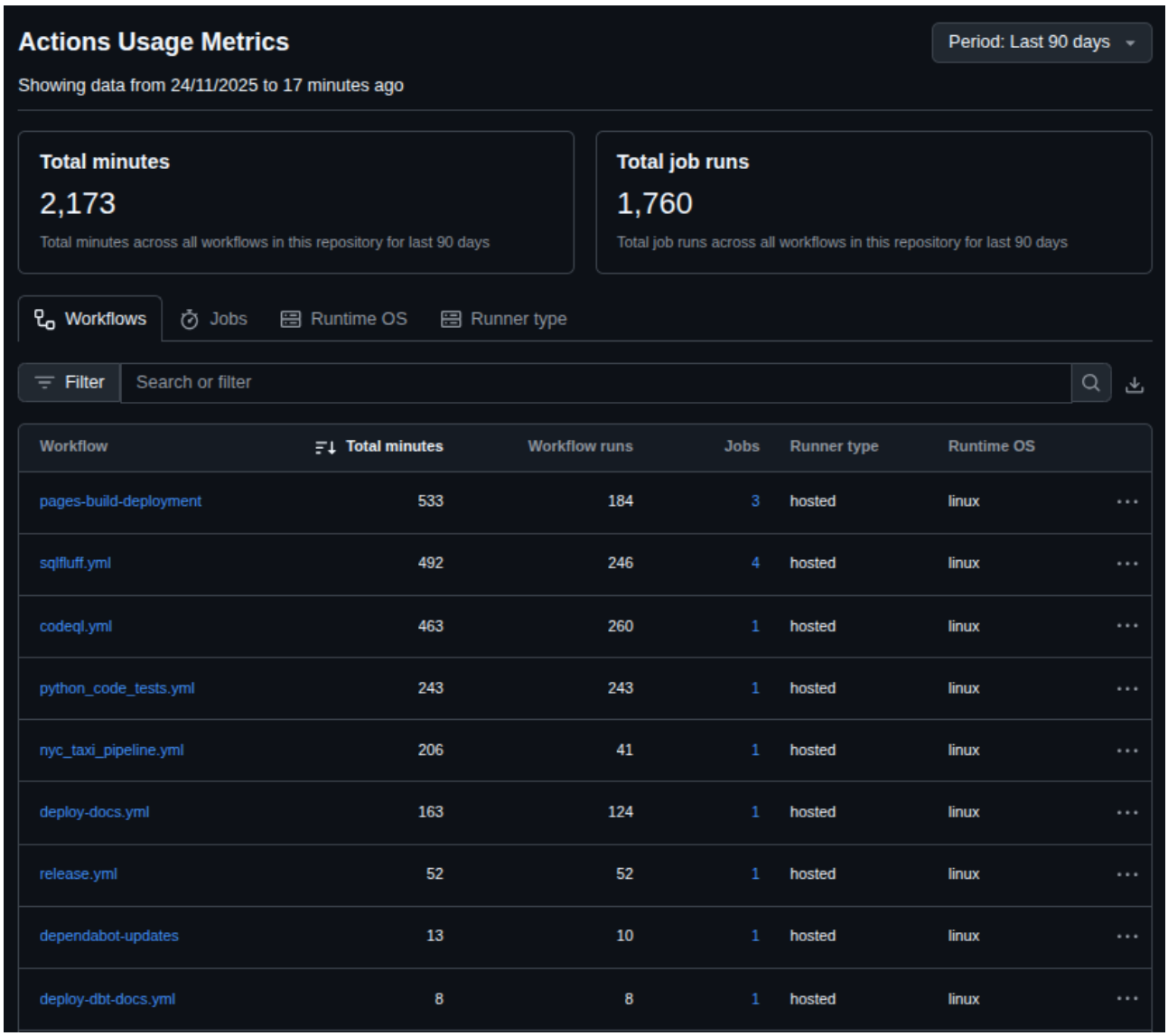
Les pénalités sont calculées sur le coût mensuel effectif :

Disponibilité mensuelle	Pénalité
< 99,9 %	10 %
< 99,0 %	25 %
< 95,0 %	50 %

4.2 Tableau de bord et indicateurs de suivi

Le suivi des indicateurs de service repose sur cinq outils complémentaires : GitHub Actions et quatre outils natifs Snowflake.

Indicateur	Outil de mesure	Métrique
Délai d'exécution pipeline	GitHub Actions Insights	Durée du workflow
Délai d'exécution pipeline	Performance Explorer	Query overload %
Délai d'exécution pipeline	Performance Explorer	Query blocked %
Délai d'exécution pipeline	Performance Explorer	Query duration
Taux de succès du chargement	FILE_LOADING_METADATA	Ratio fichiers SUCCESS
Taux de succès du chargement	Performance Explorer	Query failures/1K
Taux de succès du chargement	Performance Explorer	Query retries/1K
Fraîcheur des données	FILE_LOADING_METADATA	Date du dernier statut SUCCESS
Fraîcheur des données	Performance Explorer	Query throughput
Temps de résolution des incidents	GitHub Actions	Écart de temps emails échec/succès
Taux de réussite des tests dbt	GitHub Actions Insights	Step dbt test
Délai d'exécution pipeline	Query History	Requêtes longues
Taux de succès du chargement	Query History	Requêtes échouées
Consommation du warehouse	WH_METERING_HISTORY	CREDITS_USED
Consommation du warehouse	WH_METERING_HISTORY	CREDITS_USED_COMPUTE
Consommation du warehouse	WH_METERING_HISTORY	CREDITS_USED_CLOUD_SERVICES
Consommation du warehouse	WH_METERING_HISTORY	CREDITS_ATT_COMPUTE_QUERIES
Consommation du warehouse	Query History	Requêtes coûteuses



Workflow	Total minutes	Workflow runs	Jobs	Runner type	Runtime OS
pages-build-deployment	533	184	3	hosted	linux
sqlfluff.yml	492	246	4	hosted	linux
codeql.yml	463	260	1	hosted	linux
python_code_tests.yml	243	243	1	hosted	linux
nyc_taxi_pipeline.yml	206	41	1	hosted	linux
deploy-docs.yml	163	124	1	hosted	linux
release.yml	52	52	1	hosted	linux
dependabot-updates	13	10	1	hosted	linux
deploy-dbt-docs.yml	8	8	1	hosted	linux

Figure 6: GitHub Actions Insights : consommation par workflow

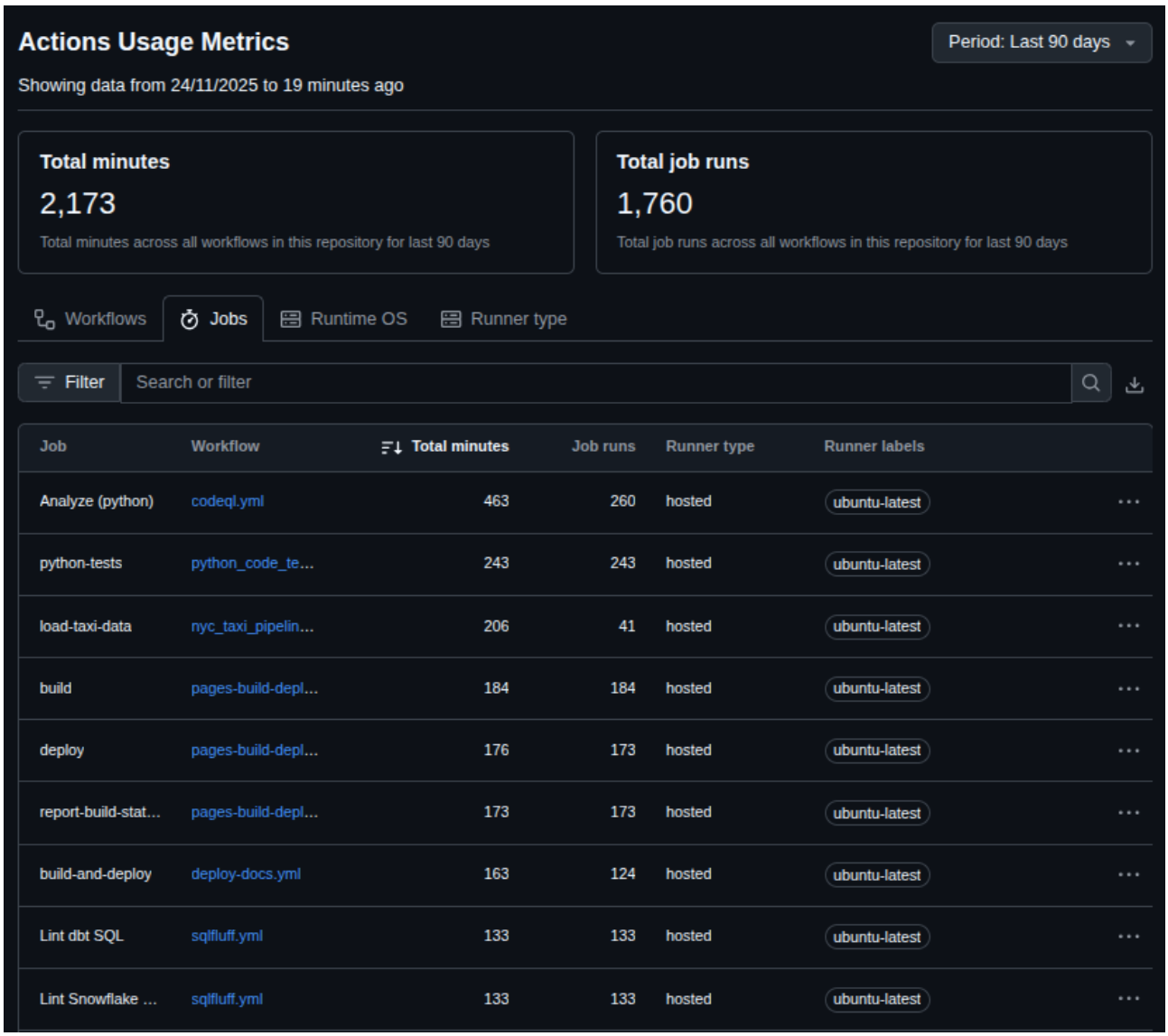


Figure 7: GitHub Actions Insights : consommation par job

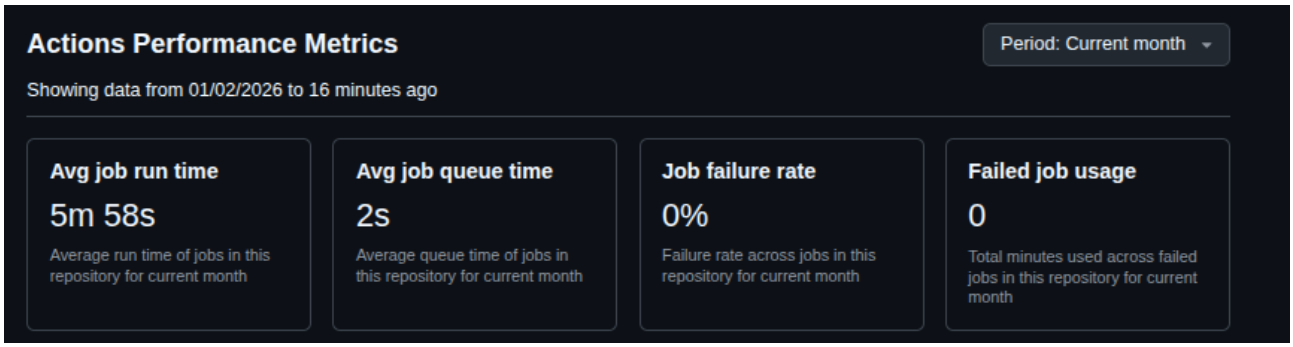


Figure 8: GitHub Actions Insights : temps moyen d'exécution et taux d'échec

WAREHOUSE_ID	WAREHOUSE_NAME	CREDITS_USED	CREDITS_USED_COMPUTE	CREDITS_USED_CLOUD_	CREDITS_ATTRIBUTED_
22	NYC_WH	0.018383618	0.017223063	0.001160555	0.000058889
21	COMPUTE_WH	0.083654080	0.083615191	0.000038889	0.000002894
22	NYC_WH	0.326093253	0.311404085	0.014689168	0.311404085
2	SNOWFLAKE_LEARNING_	0.084027428	0.083892983	0.000134445	0.000114323
22	NYC_WH	0.321840059	0.320015614	0.001824445	0.044671451
21	COMPUTE_WH	0.164580978	0.162230139	0.002350839	0.004206152
21	COMPUTE_WH	0.000005278	0.000000000	0.000005278	null
21	COMPUTE_WH	0.001116391	0.000000000	0.001116391	0.000000000
0	CLOUD_SERVICES_ONLY	0.000062500	0.000000000	0.000062500	0.000000000

Figure 9: Métriques de consommation du warehouse NYC_WH via WAREHOUSE_METERING_HISTORY

Figure 10: Métriques Performance Explorer du warehouse NYC_WH dans Snowflake

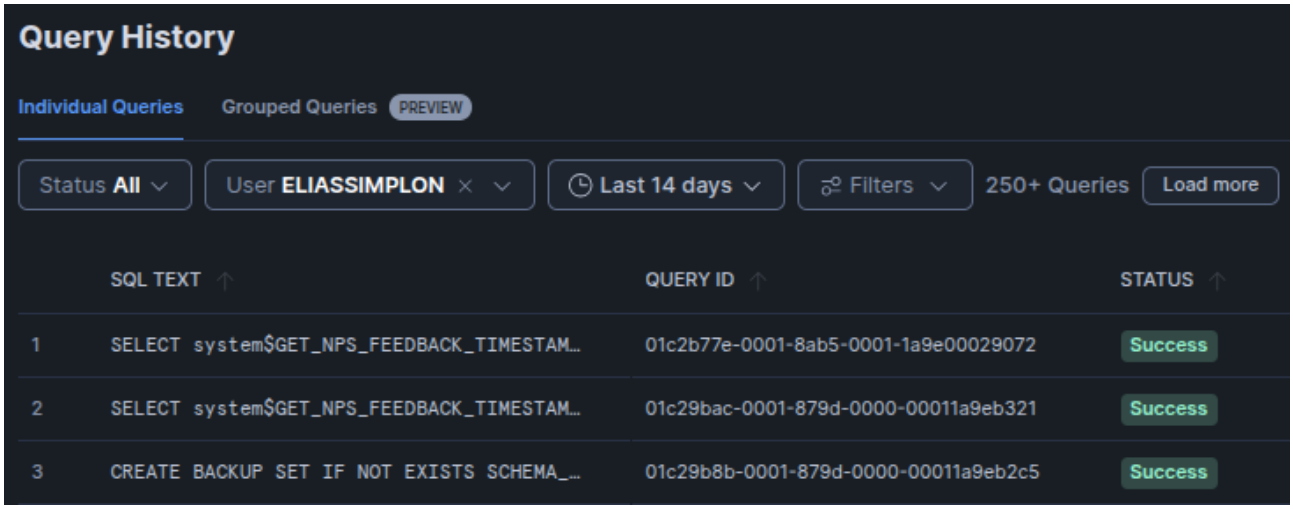


Figure 11: Historique des requêtes Snowflake — Query History

5. Backups

Le Time Travel Snowflake permet de récupérer des données supprimées ou modifiées par erreur, mais ne constitue pas un mécanisme de sauvegarde : aucune copie indépendante n'est créée et la rétention

est trop courte pour couvrir des incidents détectés tardivement. Les politiques de backup ci-dessous assurent une rétention longue durée sur des copies distinctes des données.

Les politiques de backup sont configurées automatiquement par `backup_policy.py` à la fin de chaque exécution du pipeline, mensuellement le 1er à 12h UTC. Les durées de rétention par défaut ont été choisies selon la nature et le coût de régénération de chaque objet.

Base complète (180 jours) : point de restauration global permettant de revenir à un état cohérent de tout le système en cas de catastrophe. 6 mois couvrent plusieurs cycles de production et laissent le temps de détecter une anomalie avant expiration.

Table RAW (730 jours) : les données sources sont scrapées mensuellement et cumulatives — si le site NYC TLC ne conserve plus l'historique, elles deviennent irrécupérables. 2 ans permettent un audit ou un rejeu sur plusieurs années sans dépendance externe.

Schéma FINAL (90 jours) : les tables transformées (faits et dimensions) sont reconstructibles depuis RAW via dbt. 3 mois constituent une durée standard de reprise d'activité après incident, avec un coût de stockage maîtrisé.

Ces durées sont ajustables via les variables GitHub Actions `FULL_BACKUP_POLICY_DAYS`, `RAW_TABLE_BACKUP_POLICY_DAYS` et `FINAL_SCHEMA_BACKUP_POLICY_DAYS`.

	name	database_name	schema_name	owner	owner_role_type	comment	schedule	expire_after_days	has_r
15.649 -080(	NYC_TAXI_DW_POLICY	NYC_TAXI_DW	PUBLIC	SYSADMIN	ROLE	Full Backup Policy for database NYC_TAXI_DW	USING CRON 0 12 1 * * UTC	180	false
16.059 -080(	SCHEMA_FINAL_POLICY	NYC_TAXI_DW	SCHEMA_FINAL	SYSADMIN	ROLE	Backup Policy for schema FINAL	USING CRON 0 12 1 * * UTC	90	false
15.849 -080(	YELLOW_TAXI_TRIPS_RAW_F	NYC_TAXI_DW	SCHEMA_RAW	SYSADMIN	ROLE	Backup Policy for table YELLOW_TAXI_TRIPS_RA	USING CRON 0 12 1 * * UTC	730	false

Figure 12: Politiques de sauvegarde configurées dans Snowflake

	name	database_name	schema_name	object_kind	object_name	object_database_name	object_schema_name	backup_policy_name	backup_policy_database_name	ba
)(	NYC_TAXI_DW_BACKUP	NYC_TAXI_DW	PUBLIC	DATABASE	NYC_TAXI_DW			NYC_TAXI_DW_POLICY	NYC_TAXI_DW	PUBL
)(	SCHEMA_FINAL_BACKUP	NYC_TAXI_DW	SCHEMA_FINAL	SCHEMA	SCHEMA_FINAL	NYC_TAXI_DW		SCHEMA_FINAL_POLICY	NYC_TAXI_DW	SCHE
)(	YELLOW_TAXI_TRIPS_RAW_B/	NYC_TAXI_DW	SCHEMA_RAW	TABLE	YELLOW_TAXI_TRIPS	NYC_TAXI_DW	SCHEMA_RAW	YELLOW_TAXI_TRIPS_RAW_F	NYC_TAXI_DW	SCHE

Figure 13: Affectation des politiques de sauvegarde aux objets

6. Documentation du cas d'usage de gestion

La documentation technique du projet est disponible en ligne : https://eliasmez.github.io/nyc-taxi-pipeline/

6.1 Intégration d'une nouvelle source de référence : `taxi_zone_lookup`

NYC TLC fournit un référentiel CSV de 265 zones de taxi comprenant les colonnes LocationID, Borough, Zone et service_zone. Cette donnée est stable. Elle n'est pas ingérable via le pipeline mensuel Parquet. Elle est intégrée via un **seed dbt** (`taxi_zone_lookup.csv`), chargée directement dans Snowflake via `dbt seed` et matérialisée dans le schéma RAW sous l'alias `taxi_zone_raw`.

6.2 Évolution structurelle de l'entrepôt

L'intégration du seed déclenche une chaîne de modifications : création de `taxi_zone_stg.sql` pour le nettoyage et le typage du seed, création de `dim_locations.sql` comme nouvelle dimension, mise à jour de `fact_trips.sql` pour que `pickup_location_id` et `dropoff_location_id` deviennent des FK via LEFT JOIN sur `dim_locations`, ajout de tests d'intégrité référentielle dans le YAML de `fact_trips`, et ajout de 3 marts géographiques : `zone_analysis`, `borough_analysis` et `revenue_analysis`.

```
Finished running 2 incremental models, 4 table models, 12 view models

Completed successfully

Done. PASS=18 WARN=0 ERROR=0 SKIP=0 NO-OP=0 TOTAL=18
```

Figure 14: Résultat dbt run après évolution : 18 modèles exécutés avec succès

6.3 Gestion des accès aux nouveaux objets

Les rôles ont été configurés à la création de l'entrepôt avec des droits portant sur l'ensemble des objets **présents et futurs** des schémas concernés. Tous les nouveaux objets sont automatiquement accessibles sans intervention supplémentaire : les tables ajoutées en RAW et STAGING sont couvertes par `TRANSFORMER`, qui dispose d'un accès complet à tous les schémas dont FINAL ; `dim_locations` est accessible à `BI_ANALYST` et `DATA_SCIENTIST` ; les 3 marts géographiques (vues) sont accessibles à `BI_ANALYST` et `MART_CONSUMER`.

6.4 Espace de stockage

L'ajout de `taxi_zone_stg` et `dim_locations` a un impact négligeable sur le stockage : `taxi_zone_stg` est une table transitoire sans Fail-safe et avec Time Travel limité à 1 jour, et `dim_locations` ne contient que 265 lignes. Les tables permanentes (RAW, FINAL) restent couvertes par les politiques de backup existantes. La consommation globale est surveillée via `SNOWFLAKE.ACCOUNT_USAGE.WAREHOUSE_METERING_HISTORY` et `TABLE_STORAGE_METRICS`.

6.5 Capacité de calcul

L'ajout d'une dimension et de 3 marts géographiques n'a pas nécessité d'augmentation de la capacité de calcul — le warehouse X-SMALL reste suffisant pour ce volume. Si l'évolution du pipeline venait à le nécessiter, la taille du warehouse est ajustable via la variable GitHub Actions `COMPUTE_SIZE`, qui accepte les valeurs X-SMALL, SMALL, MEDIUM ou LARGE, sans modification du code.

7. Conformité RGPD

7.1 Registre des traitements de données personnelles

Traitement	Données concernées	Finalité	Base légale	Durée de conservation
Ingestion des données de trajets	Pas de données directement identifiantes*	Analyse statistique de l'activité taxi	Intérêt légitime	Selon politique de backup
Stockage dans l'entrepôt	Localisations par zone (LocationID), timestamps	Analyse temporelle et géographique	Intérêt légitime	Selon politique de backup
Collecte analytics documentation	Données de navigation via Google Analytics	Mesure de l'audience de la documentation en ligne	Consentement cookies	2 mois

** Les données NYC TLC ne contiennent pas de données directement identifiantes : aucun nom, aucun numéro de permis, aucune adresse. Les localisations sont exprimées en zones et non en coordonnées GPS précises.*

7.2 Procédures de tri des données personnelles

Analyse des risques de ré-identification :

- Les données sont agrégées par zone et non par coordonnées GPS précises — risque de ré-identification directe faible
- Les timestamps sont à la seconde près — un croisement avec une source externe pourrait théoriquement permettre d'associer un trajet à une personne, mais la densité du trafic NYC rend ce risque négligeable en pratique

Traitements de conformité :

Procédure	Fréquence	Automatisation
Vérification de l'absence de données directement identifiantes dans les nouvelles sources	À chaque intégration de nouvelle source	Manuelle
Purge des données au-delà de la durée de rétention	Continue (via politiques de rétention)	Automatique (Snowflake)
Revue du registre des traitements	À chaque modification du périmètre	Manuelle

7.3 Cookies de la documentation en ligne (Google Analytics)

La documentation MkDocs intègre deux types de cookies tiers, gérés via le mécanisme de consentement natif de MkDocs Material.

Google Analytics (GA4) collecte des données de navigation après consentement explicite : utilisateurs actifs, pages vues, durée d'engagement, événements de navigation, géolocalisation et évaluations de pages. La rétention des données est de 2 mois. Aucun tracking n'est actif avant acceptation.

GitHub fournit le widget d'affichage du dépôt. Il ne collecte pas de données personnelles sur le visiteur — c'est un cookie fonctionnel d'affichage. Lorsque les cookies sont refusés, le widget ne se charge pas et les statistiques du dépôt ne sont pas visibles.

Le bandeau est affiché au premier chargement avec trois actions : `Accept`, `Manage` pour un choix granulaire par cookie, et `Reject`. Le choix est mémorisé dans le navigateur et modifiable à tout moment via le lien *"Change cookie settings"* en pied de page.

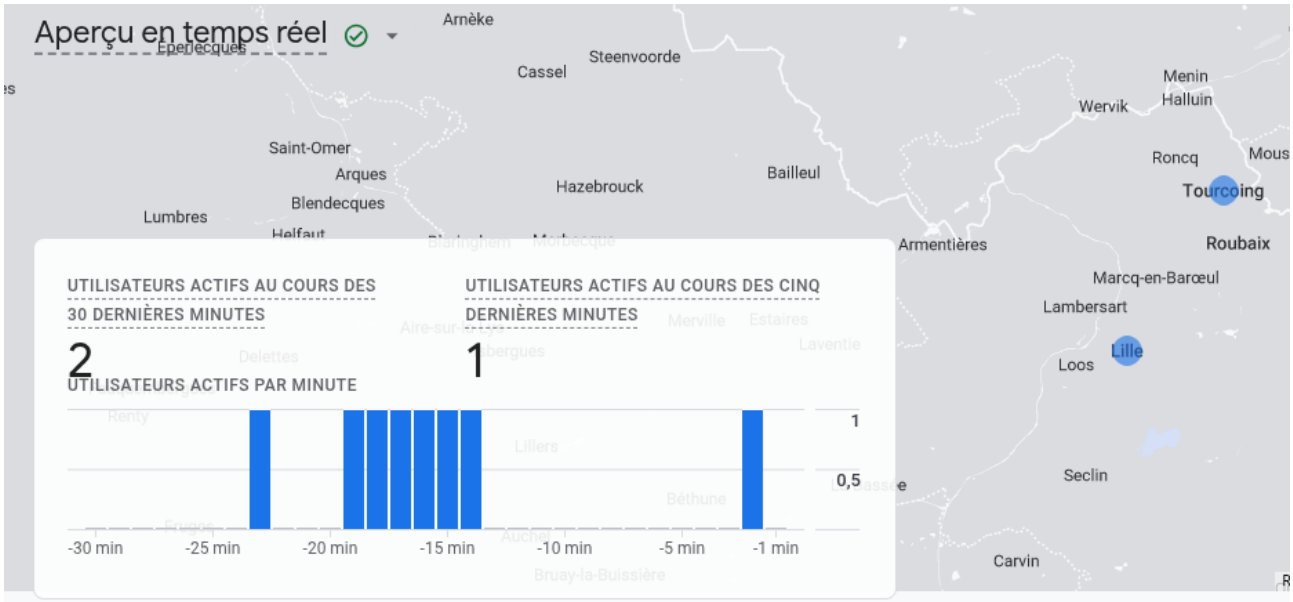


Figure 15: Vue en temps réel Google Analytics

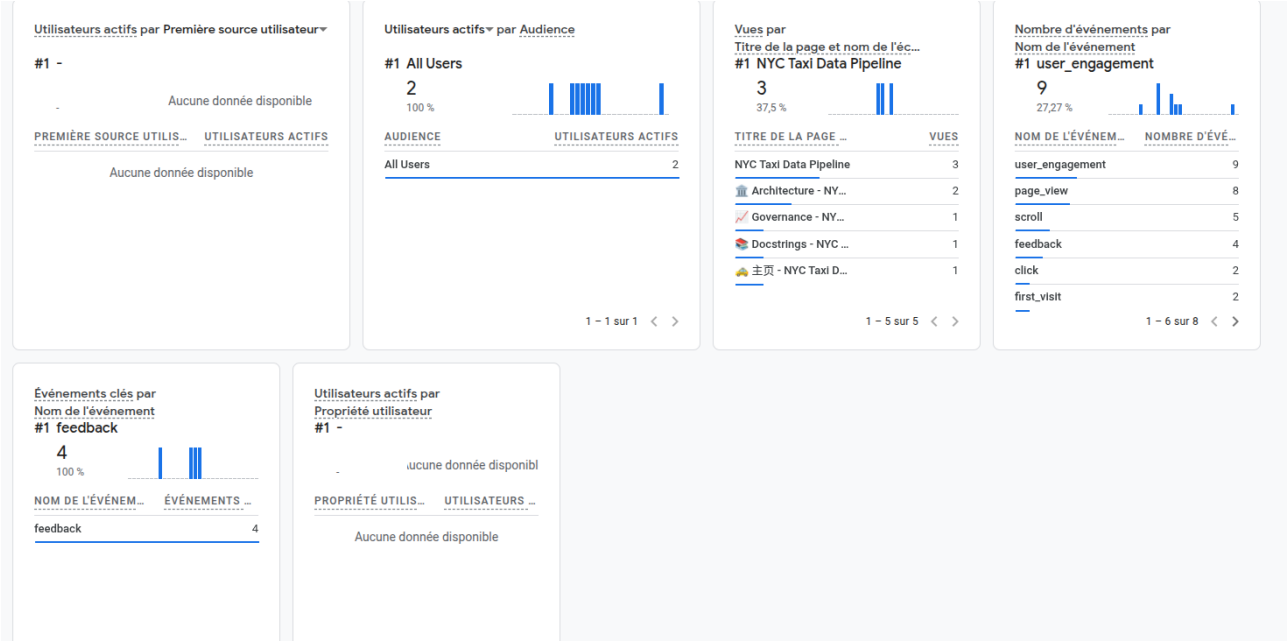


Figure 16: Tableau de bord Google Analytics

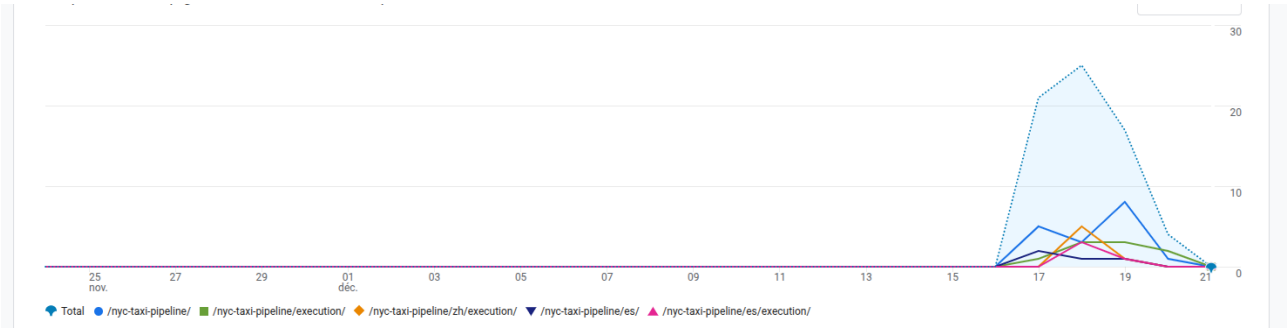
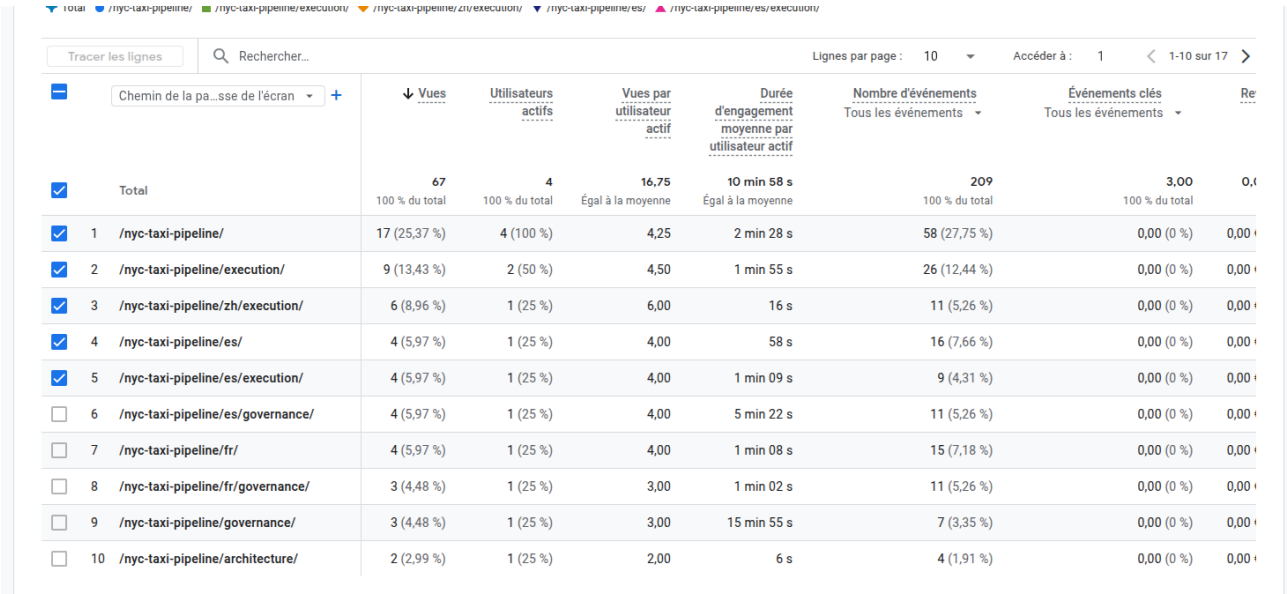


Figure 17: Évolution des pages vues dans le temps



Tracer les lignes | Rechercher... | Lignes par page : 10 | Accéder à : 1 | 1-10 sur 17

	Chemin de la pa...sse de l'écran	Vues	Utilisateurs actifs	Vues par utilisateur actif	Durée d'engagement moyenne par utilisateur actif	Nombre d'événements (Tous les événements)	Événements clés (Tous les événements)	Re
	Total	67 100 % du total	4 100 % du total	16,75 Égal à la moyenne	10 min 58 s Égal à la moyenne	209 100 % du total	3,00 100 % du total	0,
1	/nyc-taxi-pipeline/	17 (25,37 %)	4 (100 %)	4,25	2 min 28 s	58 (27,75 %)	0,00 (0 %)	0,00
2	/nyc-taxi-pipeline/execution/	9 (13,43 %)	2 (50 %)	4,50	1 min 55 s	26 (12,44 %)	0,00 (0 %)	0,00
3	/nyc-taxi-pipeline/zh/execution/	6 (8,96 %)	1 (25 %)	6,00	16 s	11 (5,26 %)	0,00 (0 %)	0,00
4	/nyc-taxi-pipeline/es/	4 (5,97 %)	1 (25 %)	4,00	58 s	16 (7,66 %)	0,00 (0 %)	0,00
5	/nyc-taxi-pipeline/es/execution/	4 (5,97 %)	1 (25 %)	4,00	1 min 09 s	9 (4,31 %)	0,00 (0 %)	0,00
6	/nyc-taxi-pipeline/es/governance/	4 (5,97 %)	1 (25 %)	4,00	5 min 22 s	11 (5,26 %)	0,00 (0 %)	0,00
7	/nyc-taxi-pipeline/fr/	4 (5,97 %)	1 (25 %)	4,00	1 min 08 s	15 (7,18 %)	0,00 (0 %)	0,00
8	/nyc-taxi-pipeline/fr/governance/	3 (4,48 %)	1 (25 %)	3,00	1 min 02 s	11 (5,26 %)	0,00 (0 %)	0,00
9	/nyc-taxi-pipeline/governance/	3 (4,48 %)	1 (25 %)	3,00	15 min 55 s	7 (3,35 %)	0,00 (0 %)	0,00
10	/nyc-taxi-pipeline/architecture/	2 (2,99 %)	1 (25 %)	2,00	6 s	4 (1,91 %)	0,00 (0 %)	0,00

Figure 18: Détail des pages visitées et métriques d'engagement

Implémenter des variations dans les dimensions de l'entrepôt de données

1. Modélisation de l'entrepôt après évolution

L'entrepôt de données comporte 3 dimensions : `dim_date`, `dim_time` et `dim_locations`.

Les dimensions `dim_date` et `dim_time` sont des données statiques par nature, les heures et les dates ne changent pas. `dim_locations` a été ajoutée comme **évolution structurelle** lors de la gestion de l'entrepôt : elle est construite depuis le seed `taxi_zone_lookup` qui provient du référentiel officiel des zones NYC TLC.

Modèle Conceptuel de Données (MCD) :

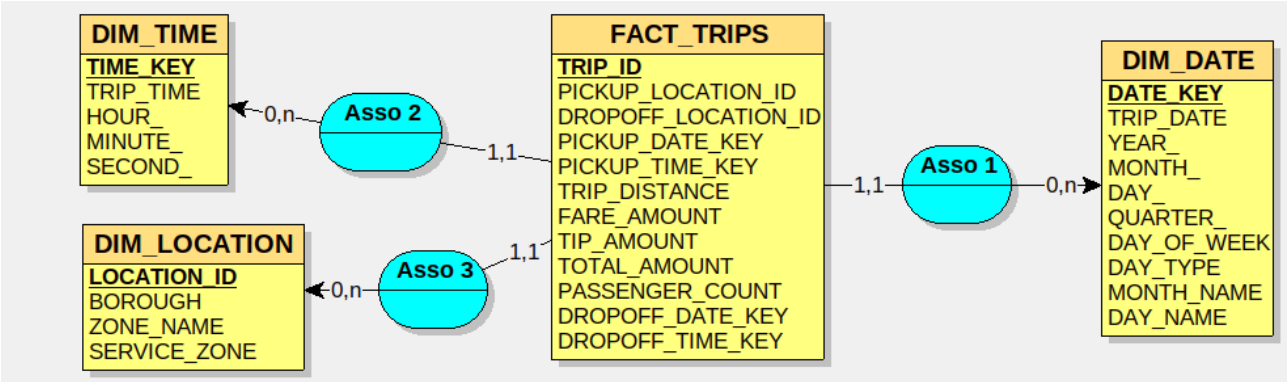


Figure 19: MCD de l'entrepôt NYC_TAXI_DW après ajout de dim_locations

Modèle Logique de Données (MLD) :

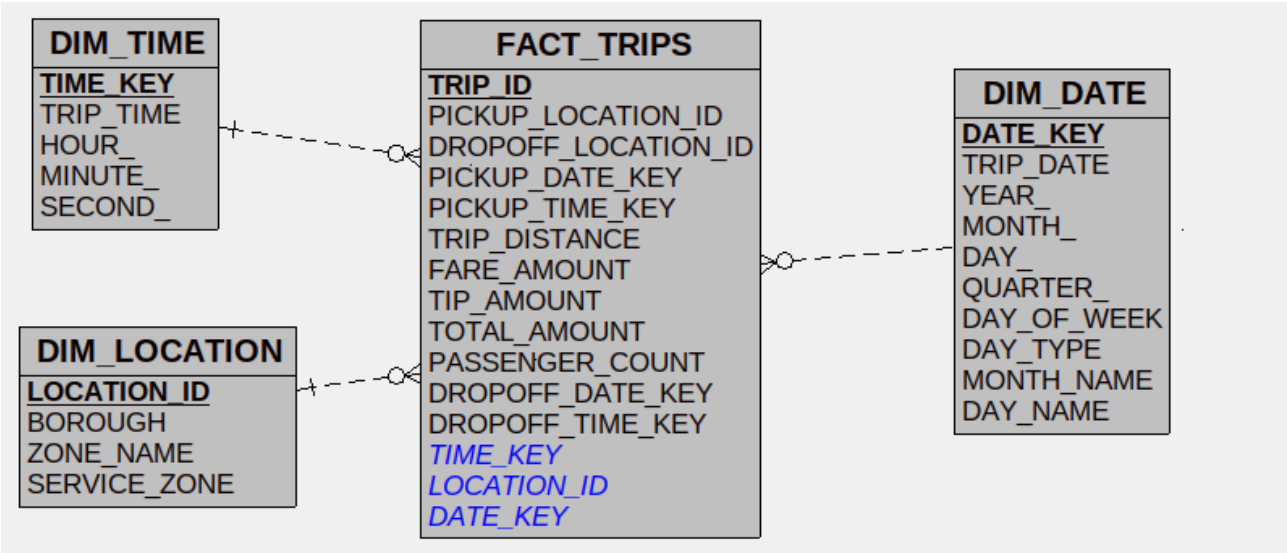


Figure 20: MLD de l'entrepôt NYC_TAXI_DW après ajout de dim_locations

Modèle Physique de Données (MPD) :

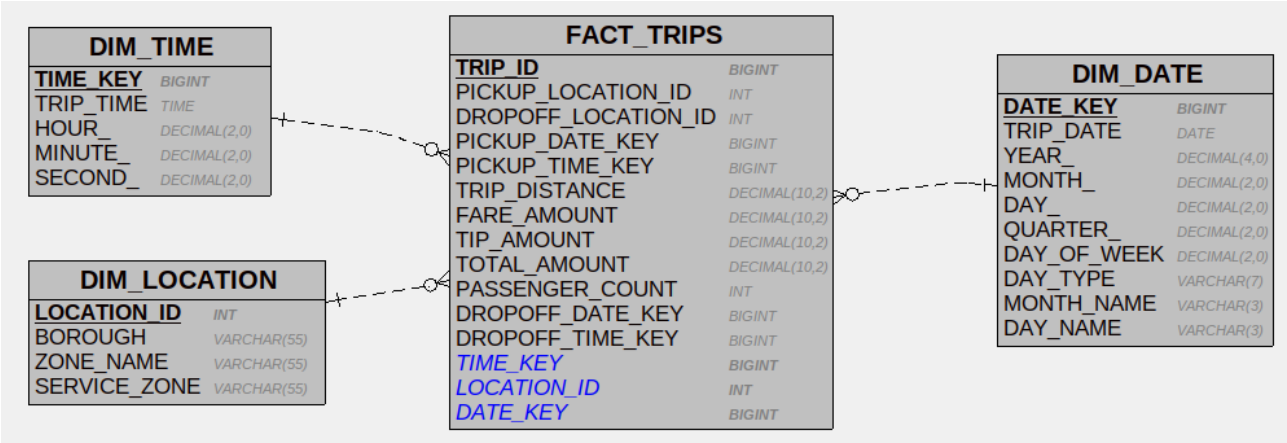


Figure 21: MPD de l'entrepôt NYC_TAXI_DW après ajout de dim_locations

---

2. Types de variations (Slowly Changing Dimensions - SCD)

2.1 Rappel des types SCD

Type	Comportement	Historisation
SCD Type 0	Aucune modification	Non
SCD Type 1	Écrasement de l'ancienne valeur	Non
SCD Type 2	Nouvelle ligne avec versioning	Oui
SCD Type 3	Colonne supplémentaire contenant l'ancienne valeur	Partielle
SCD Type 6	Ligne et colonne supplémentaire (Type 2 et 3)	Oui

2.2 Choix des SCD par dimension

Les trois dimensions sont en **SCD Type 0** : aucune variation n'est attendue, aucun historique n'est nécessaire.

dim_date et dim_time : Les attributs d'une date ou d'une heure ne changent jamais.

dim_locations : Le référentiel des zones NYC TLC est stable. Les quartiers et zones de New York ne sont pas amenés à changer dans le cadre normal d'exploitation.

2.3 Scénarios hypothétiques pour dim_locations

Bien que dim_locations soit actuellement en SCD Type 0, deux scénarios hypothétiques illustrent comment le type SCD évoluerait si les données sources changeaient :

Scénario 1 — Correction d'un nom de zone : SCD Type 1 Si une zone était mal orthographiée, la valeur serait écrasée sans historisation. Une correction orthographique n'affecte pas la cohérence des analyses passées.

Scénario 2 — Fusion ou scission de zones : SCD Type 2 Si NYC TLC scindait une zone en deux, une nouvelle ligne serait insérée avec les colonnes `valid_from DATE`, `valid_to DATE` et `is_current BOOLEAN` dans `dim_locations`, conservant l'ancienne zone pour maintenir la cohérence des trajets historiques.

3. Documentation de l'état actuel

La documentation technique est à jour et reflète l'état actuel de l'entrepôt, incluant l'ajout de `dim_locations` et les politiques SCD appliquées :

- Documentation projet : https://eliasmez.github.io/nyc-taxi-pipeline/
- dbt docs (modèles, tests, lineage) : https://eliasmez.github.io/nyc-taxi-pipeline/dbt/

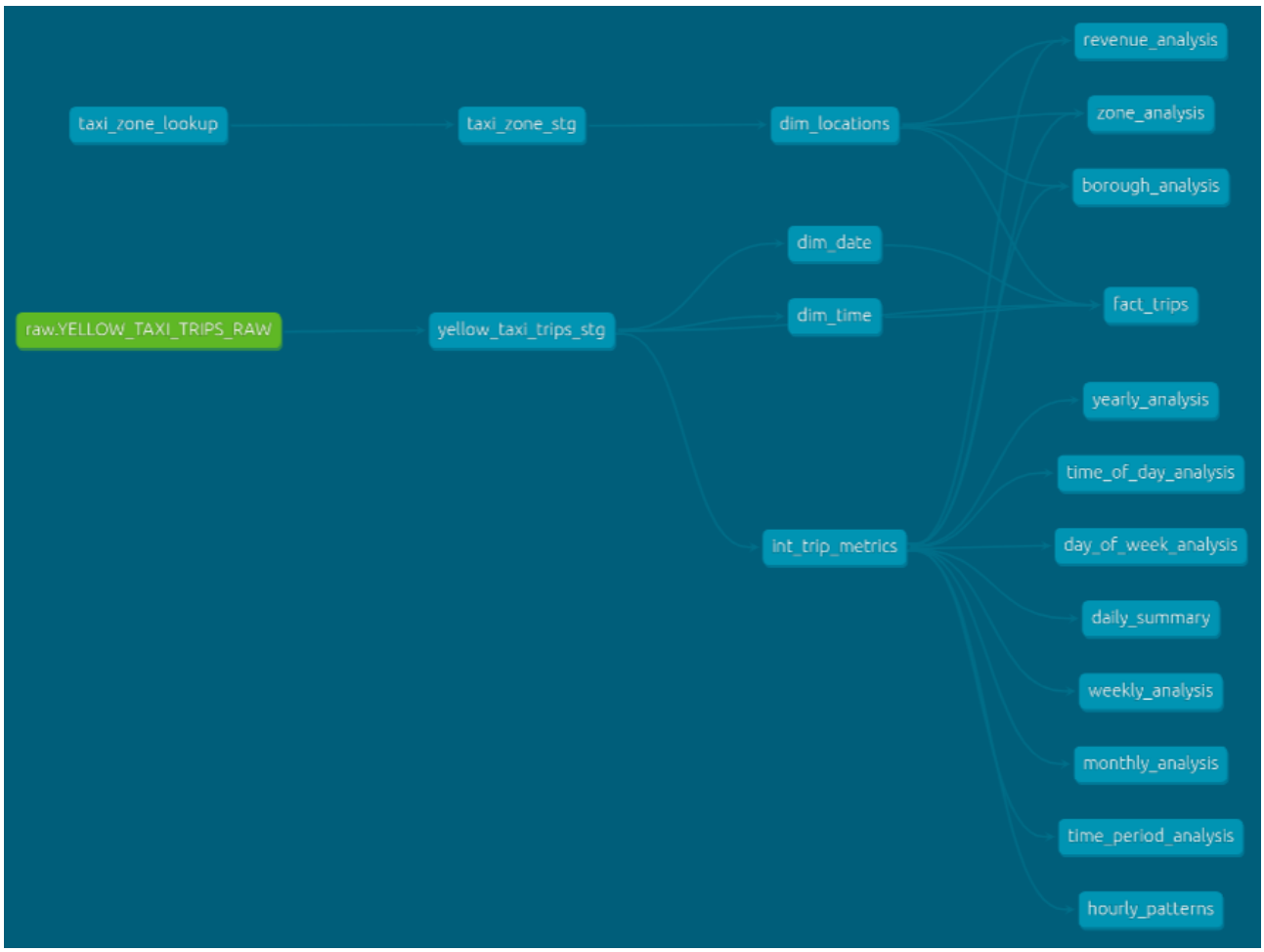


Figure 22: Lineage dbt complet : seed → staging → dimensions → faits → marts (10 marts)

Synthèse

La gestion de l'entrepôt de données NYC_TAXI_DW repose sur une **journalisation multi-niveaux** configurable via la variable `LOGGER_LEVEL`, un **suivi en temps réel de l'ingestion** via la table `FILE_LOADING_METADATA`, et des **alertes email automatiques** via GitHub Actions en cas d'échec ou d'annulation du pipeline. Les **tâches de maintenance** sont priorisées en 4 niveaux selon des objectifs mesurables et des exigences non négociables. Les **indicateurs de service** s'appuient sur les deux seuils de disponibilité du SLA Snowflake fournisseur, chacun mesurable via cinq outils dédiés. Les **backups** sont automatisés avec 3 politiques différenciées selon la nature et le coût de régénération de chaque objet. Le **cas d'usage** documente l'intégration du seed `taxi_zone_lookup`, l'évolution structurelle de l'entrepôt et l'ajout de 3 marts géographiques, avec gestion des accès et de la capacité de calcul. Concernant le **RGPD**, les données NYC TLC ne contiennent pas de données directement identifiantes ; le registre des traitements et les procédures de conformité sont en place. Les **3 dimensions** sont toutes en **SCD Type 0** — les données sont statiques par nature. Des scénarios hypothétiques illustrent les évolutions possibles : SCD Type 1 pour une correction orthographique, SCD Type 2 pour une scission de zones. La documentation est à jour et accessible en ligne.